



OptionsBleed - The Apache HTTP Server Now Bleeds

Found in the Apache HTTP server, CVE-2017-9798 lies in how Apache handles certain settings in its configuration files, resulting in memory leaks. This vulnerability is named OptionsBleed, based on its similarities with the Heartbleed vulnerability.

By: Pavan Thorat, William Gamazo Sanchez

September 22, 2017

Read time: 7 min (1757 words)



Updated on September 27, 2017, 4:00 PM PDT to add a technical analysis of the vulnerability.

A new vulnerability in the Apache HTTP server was found recently. Designated as CVE-2017-9798, this vulnerability lies in how Apache handles certain settings in its configuration files, resulting in memory leaks. This vulnerability is named OptionsBleed, based on its similarities with the **Heartbleed** vulnerability. Patches to Apache are now available.

What is OptionsBleed?

A use-after-free bug exists in the Apache HTTP Server when it handles certain settings in `.htaccess` configuration file(s). When an attacker sends an unauthenticated HTTP request with the `OPTIONS` method to a vulnerable Apache server, the server reveals some secret data from the memory in response to the request.



information leaked from the memory.

```
HTTP/1.1 200 OK
Date: Wed, 20 Sep 2017 09:25:43 GMT
Server: Apache/2.2.3 (Red Hat)
Allow: GET, HEAD, OPTIONS, HEAD, HEAD, POST, TRACE
Content-Length: 0
Connection: close
Content-Type: httpd/unix-directory
```

Figure 1. Memory leak resulting from OPTIONS request

As per Hanno Böck (the researcher who found this vulnerability) and Apache developer Jacob Champion, Apache supports a configuration directive *Limit* that restricts access to certain HTTP methods to specific users. If one sets this directive in a *.htaccess* file for an HTTP method that's not globally registered with the server, the corruption occurs. We were able to reproduce this vulnerability when the value for *Limit* directive is set to an invalid (or custom) HTTP method in the *.htaccess* file.

While doing further debugging, we noticed that the *OPTIONS* method is not needed to exploit this bug. **Any HTTP request that forces the server to send an *Allow* header in the response can be used to trigger the vulnerability.**

One common example of this is the *501 Method Not Implemented* response to a request with an unknown HTTP method. As shown in the screenshot below, when a non-vulnerable Apache HTTP Server receives an HTTP request with an unknown method, it responds with HTTP Status code 501 and the response will have the *Allow* header with a list of HTTP methods



```
ANY / HTTP/1.1
Host: test

HTTP/1.1 501 Method Not Implemented
Date: Thu, 21 Sep 2017 08:25:03 GMT
Server: Apache/2.2.3 (Red Hat)
Allow: GET, HEAD, POST, OPTIONS, TRACE
Content-Length: 275
Connection: close
Content-Type: text/html; charset=iso-8859-1
```

Figure 2. Correct HTTP 501 response

With a vulnerable server and a misconfigured .htaccess file, the same HTTP Request with an unknown method will leak parts of data from process memory in the *Allow* response header, as shown below.

```
ANY / HTTP/1.1
Host: test

HTTP/1.1 501 Method Not Implemented
Date: Thu, 21 Sep 2017 08:18:43 GMT
Server: Apache/2.2.3 (Red Hat)
Allow: GET, HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEA
D, , HEAD, , HEAD, , HEAD, , HEAD, OPTIONS, , HEAD, 
, HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, , HEAD, POST, TRACE
Content-Length: 275
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>501 Method Not Implemented</title>
</head><body>
<h1>Method Not Implemented</h1>
<p>ANY to / not supported.<br />
</p>
<hr>
<address>Apache/2.2.3 (Red Hat) Server at test Port 80</address>
</body></html>
```

Figure 3. Memory leak resulting from a misconfigured .htaccess file

1 Technical Details



memory is reserved and released. Internally, *httpd* maintains multiple pools—a global pool, and connection-specific pools. At the same time, *httpd* maintains a global variable named *METHODS_REGISTRY*, which is actually an *apr_hash_t* type from the APR library. All registered methods are stored in that variable, and as of version 2.4.26 they are:

```
683 AP_DECLARE(void) ap_method_registry_init(apr_pool_t *p)
684 {
685     methods_registry = apr_hash_make(p);
686     apr_pool_cleanup_register(p, NULL,
687                             ap_method_registry_destroy,
688                             apr_pool_cleanup_null);
689
690     /* put all the standard methods into the registry hash to ease the
691      * mapping operations between name and number
692      * HEAD is a special instance of the GET method and shares the same ID
693      */
694     register_our_method("GET", M_GET);
695     register_our_method("HEAD", M_GET);
696     register_our_method("PUT", M_PUT);
697     register_our_method("POST", M_POST);
698     register_our_method("DELETE", M_DELETE);
699     register_our_method("CONNECT", M_CONNECT);
700     register_our_method("OPTIONS", M_OPTIONS);
701     register_our_method("TRACE", M_TRACE);
702     register_our_method("PATCH", M_PATCH);
703     register_our_method("PROPFIND", M_PROPFIND);
704     register_our_method("PROPPATCH", M_PROPPATCH);
705     register_our_method("MKCOL", M_MKCOL);
706     register_our_method("COPY", M_COPY);
707     register_our_method("MOVE", M_MOVE);
708     register_our_method("LOCK", M_LOCK);
709     register_our_method("UNLOCK", M_UNLOCK);
710     register_our_method("VERSION-CONTROL", M_VERSION_CONTROL);
711     register_our_method("CHECKOUT", M_CHECKOUT);
712     register_our_method("CHECKIN", M_CHECKIN);
713     register_our_method("UPDATE", M_UPDATE);
714     register_our_method("LABEL", M_LABEL);
715     register_our_method("REPORT", M_REPORT);
716     register_our_method("MKWSPACE", M_MKWSPACE);
717     register_our_method("ACTIVITY", M_ACTIVITY);
718     register_our_method("BASELINE-CONTROL", M_BASELINE_CONTROL);
719     register_our_method("MKSPACE", M_MKSPACE);
720 }
721
```

Figure 4. Registered methods (Click to enlarge)

Note that *ap_method_regisry_init()* receives a pool. When *httpd* is starting up, the *received* pool is the global pool where *METHODS_REGISTRY* is allocated and filled with all predefined methods.

Once *httpd* is started, new methods can be added using the directive *<LIMIT>* defined in *.htaccess* files. When *.htaccess* files are read, the methods will be updated using the following at line 2275 of the following function:



Business



```
2250 limited_methods = ap_getmethodpool(cmd->temp_pool, arg, &argp - argp;
2251
2252 # if limited_methods[] is {}
2253 return NULL;
2254
2255 while (limited_methods[0] != 0) {
2256     char *method = ap_getword_conf(cmd->temp_pool, &limited_methods);
2257     int methodlen;
2258
2259     /* check for builtin or module registered method number */
2260     method = ap_method_number_of_method(methodlen);
2261
2262     if (methodlen == METHODLEN || !methodlen) {
2263         /* This method is controlled by <Limit>, not <Trace> */
2264     }
2265     else if (methodlen == 0) {
2266         /* Method has not been registered yet, but we have a restriction
2267          * in always allowed before method handling, so register it.
2268          */
2269         if (cmd->pool == cmd->temp_pool) {
2270             /* In .htaccess, we can't globally register new methods. */
2271             ap_register_method(cmd->pool, "Could not register method '%s'
2272                               " due to the .htaccess configuration",
2273                               method, cmd->pool->method);
2274         }
2275         method = ap_method_register(cmd->pool,
2276                                   ap_getword(cmd->pool, method));
2277     }
2278 }
```

Figure 5. Source code updating methods (Click to enlarge)

If one takes a closer look at the function, we can see that the *method* at line 2256 read from *.htaccess* files is allocated in the *cmd->temp_pool*. Later, at line 2276, it is copied to *cmd->pool*. When *ap_method_register()* is called, the global *METHODS_REGISTRY* residing in the global pool is updated. If *cmd->pool* is not in the global pool, the “method” residing at different pools will then be associated to the *cmd->pool*. If *cmd->pool* is released or cleaned, the pointer to method ends and references free memory. Any subsequent operation in *METHODS_REGISTRY* referring to the released memory will read free memory.

The memory leak

The leak happens when *httpd* builds an *ALLOW* header to send to the client. The *ALLOW* header is built in the following function:



```
1036 void *pall;
1037 apr_bech_this_t *bch, *bchp, *bchp2, *bchp3;
1038 if (bchp & bchp2 & bchp3) {
1039     apr_array_push(&bchp2, bchp3);
1040     apr_array_push(&bchp2, bchp);
1041 }
1042 }
1043
1044 /* TRACE is turned on a pre-serve basis */
1045 if (TRACE_ENABLED) {
1046     apr_array_push(&bchp2, bchp);
1047 }
1048 /* This is rather annoying. We should enforce registration of
1049    * the methods. */
1050 if (bchp & bchp2 & bchp3) {
1051     if (bchp2->method_list != NULL) {
1052         apr_array_cat(&bchp2->method_list, bchp3->method_list);
1053     }
1054 }
1055 return apr_array_pstrcat(&bchp2, allow, ".");
1056 }
```

Figure 6. Source code building the ALLOW header (Click to enlarge)

Note that in line 1056, the *apr_array_pstrcat()* function will take all methods from *METHODS_REGISTRY* and concatenate them into a final string that is returned to the client. For every element, the length of the *METHOD* is calculated internally using *strlen(method)*. Because the free memory was previously allocated, there is data in this area. How much memory is copied or leaked this way? That will depend on the location of the first NULL character when *strlen()* is called. For example, say I have defined a LIMIT option in an *.htaccess* file as follows:

<LIMIT DDDDDD> <LIMIT>

Sending the request *curl -s -X OPTIONS http://{IP address}/test/index.html* creates the internal list shown below. Note that the DDDDD option almost at the end of the list.



Business



```
REPORT
GET
LABEL
PROPFIND
UPDATE
MKACTIVITY
POST
MKWORKSPACE
LOCK
CONNECT
UNLOCK
OPTIONS
TRACE
PATCH
MERGE
PROPPATCH
BASELINE-CONTROL
HEAD
DELETE
VERSION-CONTROL
DDDDD
CHECKIN
```

```
HTTP/1.1 200 OK
Date: Fri, 22 Sep 2017 17:33:08 GMT
Server: Apache/2.4.26 (Unix)
Allow: GET,HEAD,POST,OPTIONS,HEAD,HEAD,TRACE
Content-Length: 0
```

Figure 7. Internal list and HTTP response

If the request is sent again, we can see the internal list at the left (note the added `\xd0`) and the HTTP response to the client at right (note the `OPTION` 7 in the `ALLOW` header):



Business



```
REPORT
GET
LABEL
PROPFIND
UPDATE
MKACTIVITY
POST
MKWORKSPACE
LOCK
CONNECT
UNLOCK
OPTIONS
TRACE
PATCH
MERGE
PROPPATCH
BASELINE-CONTROL
HEAD
DELETE
VERSION-CONTROL
\xd0}
DDDDD
CHECKIN
```

```
HTTP/1.1 200 OK
Date: Fri, 22 Sep 2017 17:33:18 GMT
Server: Apache/2.4.26 (Unix)
Allow: GET,HEAD,POST,OPTIONS,HEAD,HEAD,\x},HEAD,TRACE
Content-Length: 0
```

Figure 8. Internal list and HTTP response

It's interesting to see how the method DDDDD was added to the list again. That is because once the initial memory becomes corrupted, *httpd* will add the method again at runtime. For example, sending the request again results in another leak:

```
HTTP/1.1 200 OK
Date: Fri, 22 Sep 2017 17:46:17 GMT
Server: Apache/2.4.26 (Unix)
Allow: GET,HEAD,POST,OPTIONS,HEAD,HEAD,\x},HEAD,\x},HEAD,TRACE
Content-Length: 0
```

Figure 9. HTTP response



- ## Conditions to create the leak

```
curl -sI -X ANYANY http://192.168.93.207/test/
index.html
```

```
HTTP/1.1 501 Not Implemented  
Date: Fri, 22 Sep 2017 18:08:04 GMT  
Server: Apache/2.4.26 (Unix)  
Allow: GET,HEAD,OPTIONS,,HEAD,,HEAD,,=,HEAD,,HEAD,,=,HEAD,,=,HEAD,,=,HEAD,POST,HEAD,HEAD,TRACE  
Content-Length: 209  
Connection: close  
Content-Type: text/html; charset=iso-8859-1
```

Figure 10. HTTP response

How big can the leak be?

The size of the memory leak depends on multiple factors:



3. The maximum length of the ALLOW header.

While the second factor has a limit, the others do not. In theory, the leak can be unlimited. In practice, there's a high probability that a NULL character will stop the leak, but there is another factor to consider. Apache inspects the ALLOW header before returning it to the client. It is inspected by the ending character as specified in RFC 7230 section 3.2. This is done in the following code:

```
1596 /* Scan a string for HTTP VCHAR/obs-text characters including HT and SP
1597  * (as used in header values, for example, in RFC 7230 section 3.2)
1598  * returning the pointer to the first non-HT ASCII ctrl character.
1599  */
1600 AP_DECLARE(const char *) ap_scan_http_field_content(const char *ptr)
1601 {
1602     for ( ; !TEST_CHAR(*ptr, T_HTTP_CTRL); ++ptr) ;
1603     return ptr;
1604 }
1605
```

Figure 11. Source code for inspecting header

Once the first non-HT (horizontal tab) ASCII control character is found, the following restrictions are applied:

```
719 for (pos = *val; *pos; pos = end) {
720     end = ap_scan_http_field_content(pos);
721     if (*end) {
722         if (end[0] != CR || end[1] != LF || (end[2] != ' ' &&
723             end[2] != '\t')) {
724             ap_log_error(APLOG_MARK, APLOG_ERR, 0, ctx->r, APLOGNO(02430)
725                 "Response header '%s' value of '%s' contains "
726                 "invalid characters, aborting request",
727                 name, pos);
728             return 0;
729         }
730     }
731 }
```

Figure 12. Source code for restriction



from sending requests over the network.

Patch Details

The patch stops supporting the `<LIMIT>` in `.htaccess` files, enforcing the use of the `LIMIT` directive to the `http.conf` file. New methods are only added when `cmd->temp_pool` is equal to `cmd->pool` and assumes that `cmd->pool` is the same pool from where `METHODS_REGISTRY` is created. This is true at the web server's start-up. We can see the changes in lines 35-42 in the following code snippet.

```
1  AP_CORE_DECLARE_NONSTD(int) cmd->! ap_limit_action(cmd->cmd,
2
3
4
5  const char *temp = ap_get_core_dir, "32";
6  const char *limited_method;
7  const char *cmd-temp_data;
8  ap_pool_t *limited = 0;
9  ap_pool_t *cmd_limited = cmd->limited;
10 const char *method;
11
12 if (temp == NULL) {
13     return UNEXPECTED_DIRECTIVE_CMD;
14 }
15
16 limited_method = ap_get_core_dir(cmd-temp_pool, temp, temp - temp);
17
18 if (limited_method) {
19     return missing_method(cmd-temp);
20 }
21
22 while (limited_method) {
23     char *method = ap_get_core_dir(cmd-temp_pool, limited_method);
24     const char *method;
25
26     /* check for builtin or module registered method number */
27     method = ap_method_number_of(method);
28
29     if (method == 0) {
30         return "TRACE cannot be controlled by 'limit', see traceable";
31     }
32     if (method == 0) {
33         /* method has not been registered yet, but execution restriction
34          * is always checked before method handling, so register it:
35          */
36         if (cmd-temp == cmd-temp_pool) {
37             /* In .htaccess, we can't globally register new methods. */
38             return ap_method_number_of(method);
39         }
40         method = cmd-temp;
41     }
42 }
```

Figure 13. Source code of patch

There were some inaccurate claims made about this vulnerability. One is that the leak occurs when several `OPTION` requests are sent to the server; the leak occurs right at the start. Neither is the `OPTION` request needed.

How serious is it?

In general, `.htaccess` files provide a way to make configuration changes on a per-directory basis instead of using the main server configuration file.



Business



You should avoid using `.htaccess` files completely if you have access to `httpd` main server config file. Using `.htaccess` files slows down your Apache http server. Any directive that you can include in a `.htaccess` file is better set in a **Directory** block, as it will have the same effect with better performance.

Looking at the above guidelines and considering the necessary pre-conditions, OptionsBleed doesn't appear to be a critical bug.

However, when a user does *not* have access to the main server configuration file, the `.htaccess` file can be used to set configurations specific to your website. This will be the case in typical shared web hosting environments, where ISPs do not share root access to the server system and the main server configuration file.

An attacker using a shared host can maliciously craft a `.htaccess` file for his own website and, use the vulnerability to try to collect the memory leaks. This may contain sensitive data from any other co-hosted websites. Shared hosting is commonly used by many site owners with limited requirements; OptionsBleed puts those users at risk.

Both the 2.2.x (up to 2.2.34) and the 2.4.x branches (up to 2.4.27) are affected by this vulnerability. Patches have already been released by many Linux distributions to cover this flaw, and we recommend that system administrators do so if they haven't already.

Conclusion

[Security customers can use the following DPI rule to configure



specimens released on the vulnerability has been released as follows:

- **1002593 - Allow HTTP (Including WebDAV) Methods**
- **1008618 - Apache HTTP OPTIONS Information Disclosure Vulnerability (CVE-2017-9798)**

System administrators should upgrade their installed versions of Apache HTTP Servers and disable the use of *.htaccess* files, if possible. Site owners *without* access to these directly and reliant on their ISPs should prod these service providers to upgrade as well.

TippingPoint has posted a Customer Shield Writer (CSW) file for this vulnerability that are available for customers to download on [TMC](#). The applicable rule is as follows:

- **C1000002: HTTP: Apache Server Options Information Disclosure Vulnerability**

Tags

[Network](#) | [Research](#)



Business



Pavan Thorat

Vulnerability Researcher

William Gamazo Sanchez

Sr. Threat Researcher

CONTACT US

Related Articles

[Python-Based NodeStealer Version Targets Facebook Ads Manager](#)

[Vishing via Microsoft Teams Facilitates DarkGate Malware Intrusion](#)

[Earth Koshchei Coopts Red Team Tools in Complex RDP Attacks](#)

See all articles >

**Experience our unified
platform for free**

Claim your 30-day trial



Business



Resources



Support



About Trend



Country Headquarters

Trend Micro - United States (US)

225 East John Carpenter Freeway
Suite 1500
Irving, Texas 75062

Phone: +1 (817) 569-8900

Select a country / region

United States



Privacy

Legal

Accessibility

Terms of Use

Site map

Copyright ©2024 Trend Micro Incorporated. All rights reserved